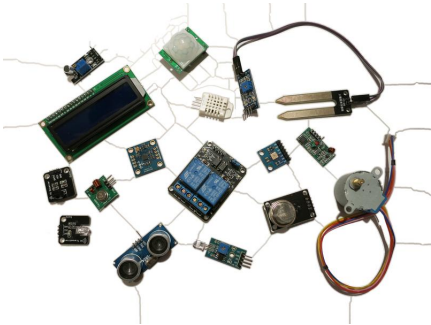




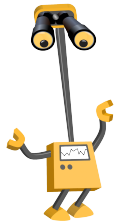
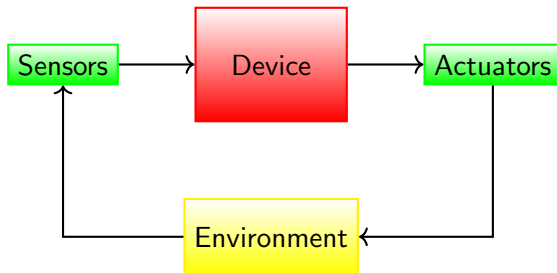
Embedded Systems

Sensors / Actuators

H. Cassé

2021

Reactive Systems



- ▶ *Open Loop* – no sensor (less adaptative)
- ▶ *Closed Loop* – sensors (more reactive)

Example: Espresso Machine

Control panel:

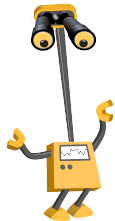
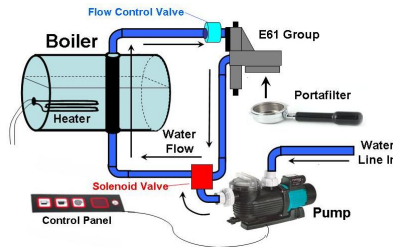
- ▶ LEDs, digital LEDs
- ▶ push buttons, potentiometer

Sensors:

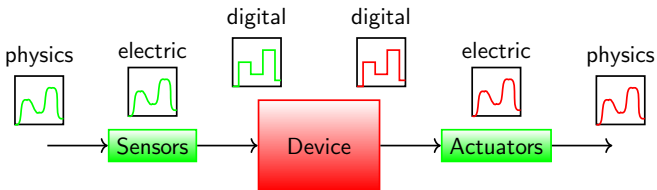
- ▶ thermistor (temperature)
- ▶ pressure

Actuators:

- ▶ pump (electric motor)
- ▶ heater (heating resistor)
- ▶ valves



Sampling



Several electric representations:

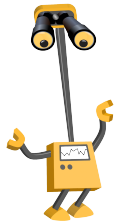
- ▶ voltage
- ▶ pulse width
- ▶ pulse frequency

Time:

- ▶ sensing time
- ▶ + computation time
- ▶ + actuating time
- ▶ < application period/frequency

Outline

- 1 Introduction
- 2 Sensor / Actuators
- 3 Conclusion



Transducer

Definition (Transducer)

A device that is actuated by power from one system and supplies power usually in another form to a second system.

From *Merriam-Webster*

Wikipedia: Device that converts energy from one form to another. Usually a transducer converts a signal in one form of energy to a signal in another

sensor Transducer from physical quantity to electric **signal**.

actuator Transducer electric **signal** to physical quantity.



Definition (Signal)

- 1 A sequence of states representing an encoded message in a communication channel.
- 2 Any variation of a quantity or change in an entity over time that conveys information upon detection.

Characteristics

Physical domain:

- ▶ type (temperature, pressure)
- ▶ range (min, max)
- ▶ resolution (smallest variation)
- ▶ sensitivity (reaction time)
- ▶ error (%)
- ▶ linearity domain

Electric Domain:

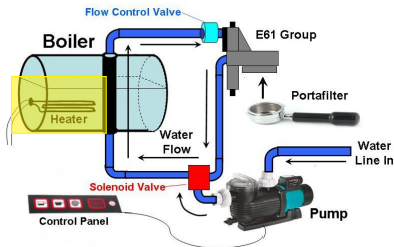
- ▶ signal type (voltage, pulse, ...)
- ▶ range (0, max)

Environment: work
condition

- ▶ temperature
- ▶ pressure
- ▶ humidity
- ▶ radiation

Question: is your sensor/actuator adapted to your application?

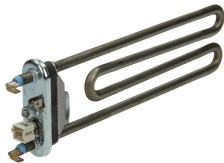
Actuator: electric heater



- ▶ domain: 0-150°C
- ▶ power: 750W
- ▶ input: 0-220V
- ▶ precision: ∞
- ▶ reaction time: slow

$$P = \frac{V^2}{R}$$

T° proportional to V^2 !
 P (digital) $\implies V$
 (voltage)



Digital-to-Analog Converter (DAC)



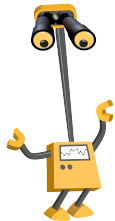
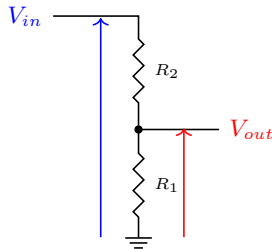
DAC Output

$$V_{out} = \frac{N}{2^n - 1} \times V_{ref}$$
$$n \in \{8, 10, 12\}$$

Voltage Divider

$$V_{out} = \frac{R_1}{R_1 + R_2} \times V_{in}$$

(Ohm's Law)



String DAC

Example: 3-bit DAC

$$N = N_2 N_1 N_0$$

Case: $N = 6 = (110)_2$

Pros: simple, fast

Cons: lot of hardware

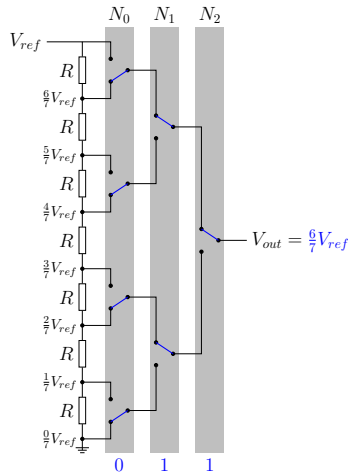
$2^n - 1$ resistors

$2^n - 1$ switches

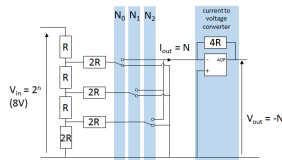
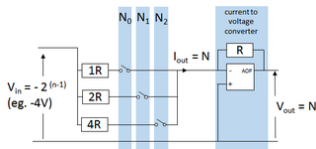
$$n = 12$$

1023 resistors

1023 switches



Improvements



Binary weighted DAC

Millman law:

$$V_{out} = 2V_{ref} \left(\frac{N_2 R}{R} + \frac{N_1 R}{2R} + \frac{N_0 R}{4R} \right)$$

$$= V_{ref} \left(\frac{4N_2 + 2N_1 + 1N_0}{8} \right)$$

 $\oplus$: n resistors + n transistors

$\ominus$: range of resistors from R to $2^n R$

R-2R DAC

 $\oplus$: $2n$ resistors + n transistors $\oplus$: resistor range R - $4R$

OpAmp: to smooth the output out.



From <https://commons.wikimedia.org/w/index.php?curid=56162128>

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ — ▶ ◀ ≡ ▶ ≡ ≡ ↺ 🔍 ↻

Programming it

Initialization (channel 1):

```
DAC_CR = DAC_EN1;
```

Setting the value 12-bit value N :

```
DAC_DHR12R1 = N;
```

$\Rightarrow \frac{N}{4096} \times 3.3V$ to DAC_OUT1

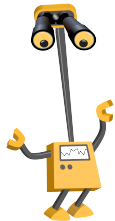
What about wiring of these outputs?

- ▶ *DAC_OUT1* wired to *GPIOA4*
- ▶ *DAC_OUT2* wired to *GPIOA5*

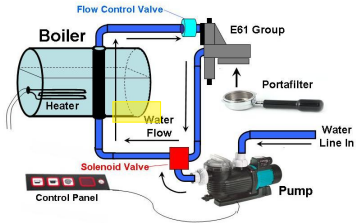
Analog mode in GPIO:

```
GPIOA_MODER = RET_BITS(GPIOA_MODER, 4*4, 4, GPIO_MODER_ANA);
```

Sensor / Actuators



Sensor: thermistor

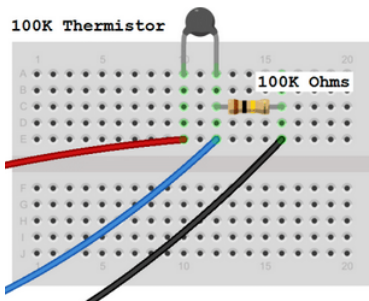
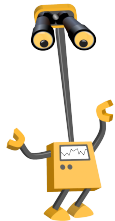


NTC: temperature $\nearrow$
 $\Rightarrow$ resistance $\searrow$
 $\Rightarrow$ voltage $\searrow$

- ▶ $50k\Omega / 25^{\circ}\text{C}$
- ▶ range: $-55^{\circ}\text{C} - 125^{\circ}\text{C}$



Converting to temperature



Divider wiring:

$$V_{out} = V_{in} \times \frac{R}{10^5 + R}$$
$$\Leftrightarrow R = 10^5 \times \left(\frac{V_{in}}{V_{out}} - 1 \right)$$

Steiner-Hart equation:

$$\frac{1}{T} = a + b \ln R + c (\ln R)^3$$

How to get a , b , c ?

Calibration (1)

- ▶ sensor building not perfect: slight sensitivity error
- ▶ precision depends on price

For Steiner-Hart: 3 measurement points (T_i, R_i)

$$\gamma_2 = \frac{\frac{1}{T_2} - \frac{1}{T_1}}{\ln R_2 - \ln R_1} \quad \gamma_3 = \frac{\frac{1}{T_3} - \frac{1}{T_1}}{\ln R_3 - \ln R_1}$$

$$c = \frac{\gamma_3 - \gamma_2}{(\ln R_3 - \ln R_2)(\ln R_1 + \ln R_2 + \ln R_3)}$$

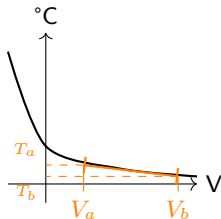
$$b = \gamma_2 - c (\ln^2 R_1 + \ln R_1 \ln R_2 + \ln^2 R_2)$$

$$c = \frac{1}{T_1} - (b + c \ln^2 R_1) \ln R_1$$



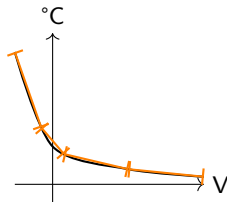
Calibration (2)

Linearity interval:



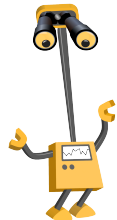
$$T = T_a + \frac{T_b - T_a}{V_b - V_a} \times V_{out}$$

Linear interpolation:



$\{(V_i, T_i)\}$
if $V_i \leq V_{out} < V_{i+1}$,

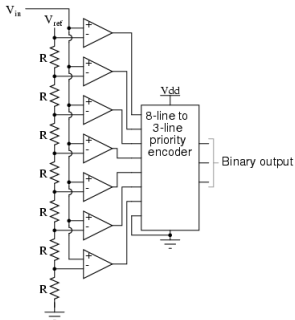
$$T = T_i + \frac{T_{i+1} - T_i}{V_{i+1} - V_i} \times V_{out}$$



Analog Digital Converter (ADC) – n -bit

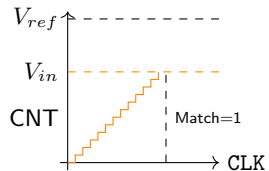
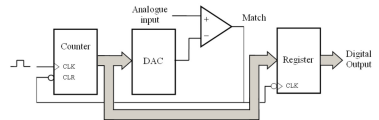


Flash ADC (voltage divider):



- ▶ speed: very fast
- ▶ cost: high
 2^n resistors + 2^n opamps

Simple ramp ADC:



- ▶ speed: $2^{n-1} \times t_{setup}$ on average
- ▶ cost: 1 DAC

ADC continued

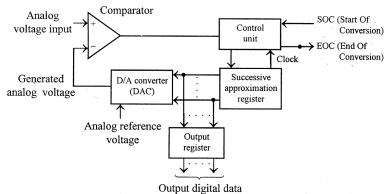
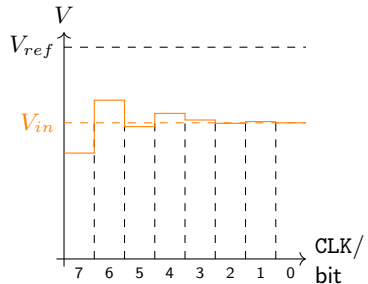


Fig 7.12 : Successive-approximation A/D converter



```

 $N \leftarrow 0 \dots 0$ 
for  $i \leftarrow n - 1$  to  $0$  do
     $N_i \leftarrow 1$ 
    if  $DAC(N) > V_{in}$  then
         $N_i \leftarrow 0$ 
    end if
end for

```

- ▶ speed: $n \times t_{setup}$ on average
- ▶ cost: 1 DAC + logic

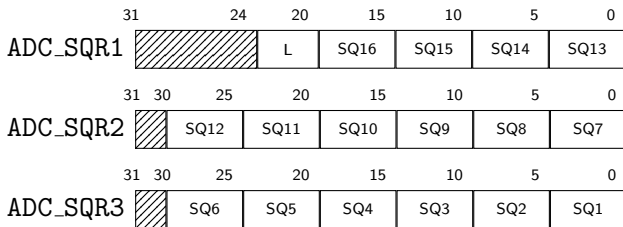


- ## Sensor / Actuators

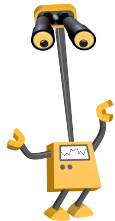
Multiple conversions

32 (5-bit) inputs

Upto 16 channels scanned in the same operation ((4-bit):



$L = \text{number of used SQ entries} - 1$



Multiple mode

On ADC1_IN3-5 (GPIOA3-5).

Initialization (12-bit):

```
for(int i = 3; i <= 5; i++)
    GPIOA_MODER = REP_BITS(GPIOA_MODER, i*2, 2, GPIO_MODER_ANA);
ADC1_SQR1 = REP_BITS(0, 20, 4, 2);
ADC1_SQR3 = REP_BITS(ADC1_SQR3, 0*5, 5, 3);
ADC1_SQR3 = REP_BITS(ADC1_SQR3, 1*5, 5, 4);
ADC1_SQR3 = REP_BITS(ADC1_SQR3, 2*5, 5, 5);
ADC1_CR1 = 0;
ADC1_CR2 = ADC_ADON;
```



Getting a measurement:

```
ADC1_CR2 |= ADC_SWSTART;
while ((ADC1_SR & ADC_EOC) == 0);
x3 = ADC1_DR;
while ((ADC1_SR & ADC_EOC) == 0);
x4 = ADC1_DR;
```

Performances

Sampling time:

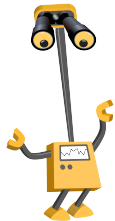
Bits	Time (in cycle)	Max freq.
12	15	560 Khz
10	13	6,5 Mhz
8	11	7,6 Mhz
6	9	9,3 Mhz



CD format: 44,1 Khz / 16-bit

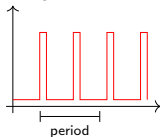
Outline

- 1 Introduction
- 2 Sensor / Actuators
- 3 Conclusion

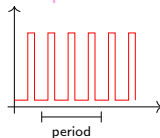


Value Encoding with Time

- Frequency: how many pulses in the period?

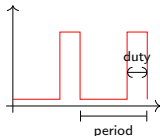


small value

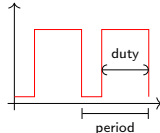


big value

- ▶ Pulse Width Modulation (PWM): how wide is the pulse?



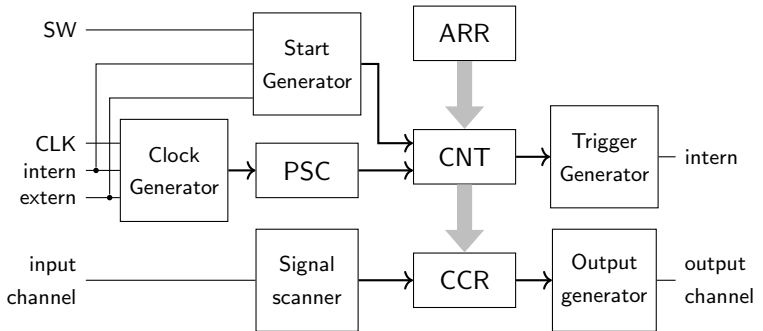
small value



big value



Timer: Swiss Army Knife



ARR = Auto Reload Register, **CNT** = CouNTer
PSC = PreSCaling, **CCR** = Capture/Compare Register

STM32F4

Timer	Counter	Channels	APB	Features
TIM_1, TIM_8	16-bit	4	2	PWM-oriented
TIM_2, TIM_5	32-bit	4	1	capture/PWM
TIM_3, TIM_4	16-bit	4	1	capture/PWM
TIM_6, TIM_7	16-bit	0	1	DAC command
TIM_9-11	16-bit	2	2	
TIM_12-14	16-bit	2	1	

Ports:

- ▶ CLK_INT – APB1_CLK or APB2_CLK
- ▶ TIMx_CHy – input/output channel
- ▶ TIMx_ETR – external trigger
- ▶ ITRx – inter-timer synchronization

APB1_CLK = 42 MHz – APB2_CLK = 84 MHz

Timer – Swiss Army Knife

Tuning the Prescaler

CNT capacity bounded:

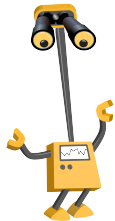
APB	Frequency	16-bit	32-bit
1	42 Mhz	1,6 ms	102 s
2	84 Mhz	0,8 ms	51,1 s

$PSC = n$: $n + 1$ pulses from CLK_INT
 $\Rightarrow$ 1 incrementation of CNT

Example: $TIM3_PSC = 1000-1 \Rightarrow$ maximum time
 $T = 2^{16} / (42,000,000 / 1,000) = 0,78s$

Time/pulse relationship:

$$\frac{N \text{ pulses}}{\text{CLK_INT/PSC pulses}} = \frac{T \text{ s}}{1 \text{ s}} \iff T = \frac{N \times \text{PSC}}{\text{CLK_INT}}$$



Best Prescaling

Best prescaler for a period T s ($N = 2^{16}$ or $N = 2^{32}$):

$$\text{PSC} = \left\lceil \frac{T \times \text{CLK_INT}}{N} \right\rceil$$

Example: TIM3 with $T = 100 \text{ ms} = 0.1 \text{ s}$

$$\text{PSC} = \left[\frac{0.1 \times 42,000,000}{65536} \right] = 65$$



Computing a duration in pulses

With a fixed PSC:

$$N = \frac{T \times \text{CLK_INT}}{\text{PSC}}$$

Example:

► $T = 100 \text{ ms}$

$$N = \frac{0.1 \times 42,000,000}{65} = 64615.38$$

Remark: $64615,38 < 2^{16} = 65536!$)

► $T = 20 \text{ ms}$

$$N = \frac{0.02 \times 42,000,000}{65} = 12923.08$$



About the resolution

Resolution = pulse number for the period T

Precision

$$T_{prec} = \frac{1 \times \text{PSC}}{\text{CLK_INT}}$$

PSC	Resolution	Precision (μs)
65	64,615.38	1,52
100	42,000	2.38
1000	4,200	23.8
128	32,812.5	3,05
84	50,000	2

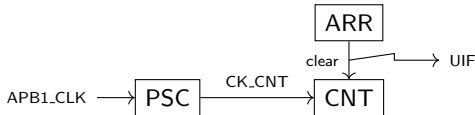


Clock skew with the timer

$$T = 0.1 \text{ s}$$

- ▶ $PSC = 65 \implies N = 64,615.38$
- ▶ $ARR = 64,615$ (rounded)
- ▶ $error = 0.38 p \implies 0,59 \mu s$
- ▶ 1 error period (0.1s) after 170,040 period
 $\iff 17,004 s (4.7 h)$

Reminder: timer to wait



```
#define WAIT_PSC          1000
#define WAIT_DELAY        (APB1_CLK / PSC)

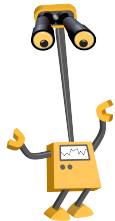
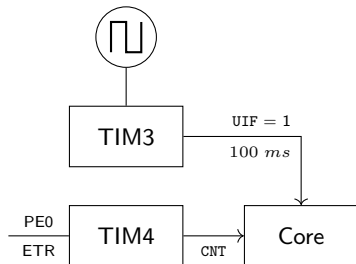
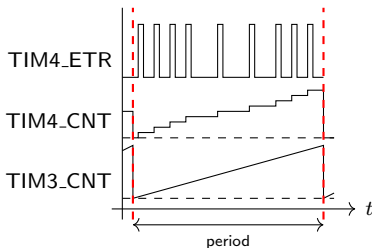
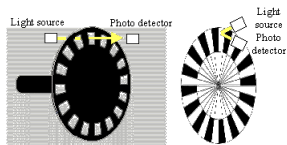
/* initialization */
TIM3_CR1 = 0;                // stop timer
TIM3_PSC = WAIT_PSC - 1;     // PSC = 0 → divide by 1
TIM3_ARR = WAIT_DELAY;

/* waiting */
TIM3_SR = 0;                // clear SR
TIM3_EGR = TIM_UG;          // reset CNT
TIM3_CR1 = TIM_CEN;         // start the timer
while ((TIM3_SR & TIM_UIF) == 0); // Update Interrupt Flag
TIM3_SR = 0;                // reset UIF
```

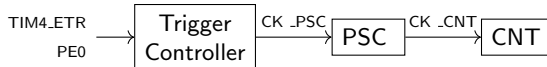


Counting the frequency

Wheel speed sensor



Programming

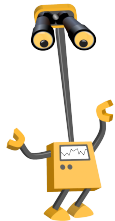


Configuration

```
TIM4_CR1 = 0;  
TIM4_PSC = 0;  
TIM4_ARR = 65535; // max count  
// Slave Mode Control Register  
TIM4_SMCR = TIM_ECE // Enable  
            | TIM_ETRF; // Ext. Clock
```

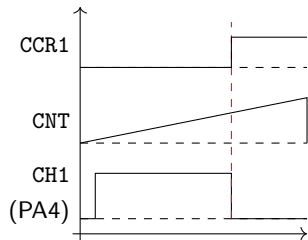
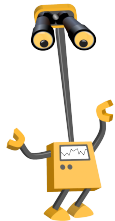
Measure

```
TIM3_SR = 0;  
TIM3_EGR = TIM_UG;  
TIM3_CR1 = TIM_CEN;  
while((TIM3_SR & TIM_UIF) == 0);  
x = TIM4_CNT;
```

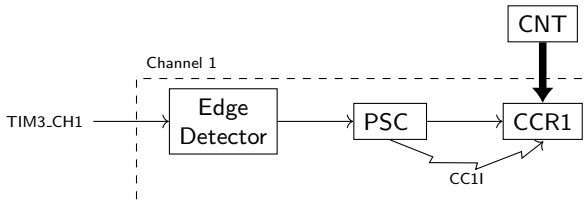


Pulse Width Capture

Sonar: distance proportional to falling edge delay



Signal Capture Block



Initialization

```
// Capture Compare Register 1
// input 1, edge rise, enable
TIM3_CCMR1 = TIM_CC1S_T11;
TIM3_CCER = | TIM_CC1P_rise
            | TIM_CC1E;

TIM3_PSC = 0;
TIM3_ARR = TIMEOUT;
```

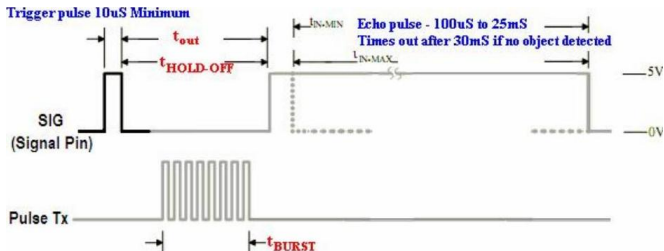
Measurement

```

GPIOB_BSRR = 1<<3; // emit on PD3
TIM3_SR = 0;
TIM3_EGR = TIM_UG;
TIM3_CCR1 = TIM_CEN;
#define WAIT_FLAGS (TIM_UIF|TIM_CC1F)
while((TIM3_SR & WAIT_FLAGS) == 0);
if((TIM3_SR & TIM_CC1F) != 0)
    x = TIM3_CCR1;
else
    x = TIMEOUT;

```

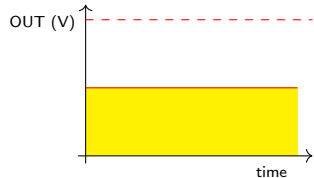
Rising and Falling Edges



```
TIM3_CCMR1 =
    TIM_CCS1S_Tt1 // input 1
    | TIM_CCS2S_Tt1 // input 1
TIM3_CCER =
    | TIM_CC1P_fall // edge
    | TIM_CC1E      // enable
    | TIM_CC2P_rise // edge
    | TIM_CC2E;     // enable
```

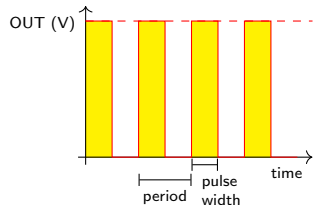
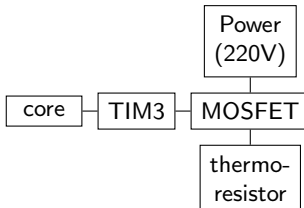
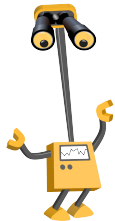
```
TIM3_SR = 0;
TIM3_EGR = TIM_UG;
TIM3_CCR1 = TIM_CEN;
#define WAIT_FLAGS (TIM_UIF|TIM_CC1F)
while((TIM3_SR & WAIT_FLAGS) == 0);
if((TIM3_SR & TIM_CC1F) != 0)
    x = TIM3_CCR1 - TIM3_CCR2;
else
    x = MAX_DIST;
```

Timer to drive an actuator



Problem: $OUT = 5\text{ V} \times 150\text{ mA} = 750\text{ mW}$

Needed: 750 W

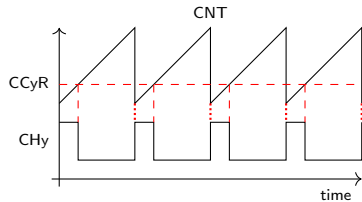
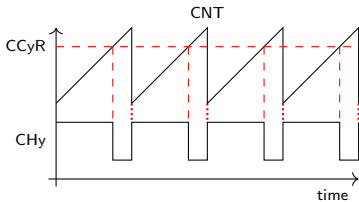
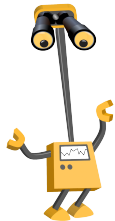
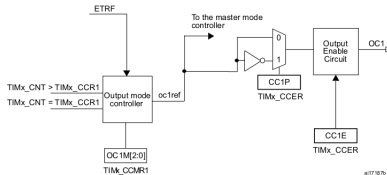


Power: relative to pulse width.

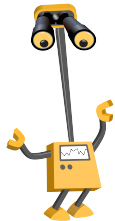
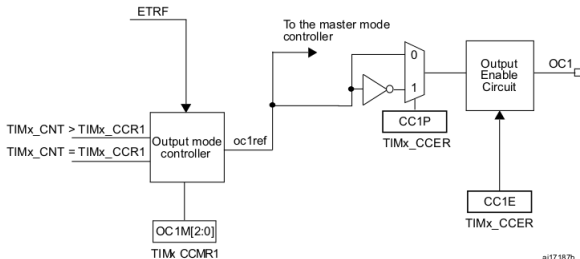
Period: device dependent (inertia)

Timer – Swiss Army Knife

Programming PWM with timer



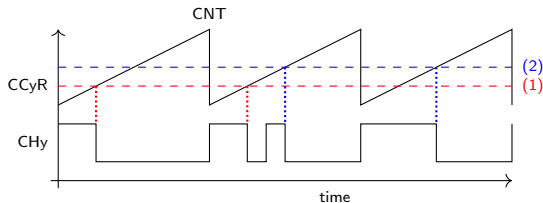
Configuring the timer



```
TIM3_CCMR1 = TIM_OC1M_PWM1;  
TIM3_CCER = TIM_CC1E;  
TIM3_CCR1 = 0;  
TIM3_PSC = THERM_PRESCALE;  
TIM3_ARR = THERM_PERIOD;
```

```
TIM3_EGR = TIM_UG;  
TIM3_CR1 = TIM_CEN;  
while(1) {  
    ...  
    TIM3_CCR1 = x;  
    ...  
}
```

Update Synchroniation



Solutions:

- ▶ Wait for update.
- ▶ Configuration for buffering:

```
TIM3_CR1 = ... | TIM_ARPE;
```

Example

Requirement for thermo-resistance: period 100 ms
 Driven by TIM3 (powered by $\text{APB1_CLK} = 43\text{ MHz}$)

$$PSC_{best} = \frac{42,000,000}{65,536} = 64,08$$

Chosen PSC: 84

$$period_{pulses} = \frac{0.1 \text{ s} \times 42,000,000}{84} = 50,000$$

Setting power at N %:

$$CCR1 = \frac{period_{pulses} \times N}{100}$$



Example (continued)

```
#define THERM_PSC 84
#define THERM_PERIOD (APB1_CLK / THERM_PSC)

void init_therm() {
    TIM3_CCMR1 = TIM_OC1M_PWM1
    TIM3_CCER = TIM_CC1E;
    TIM3_CCR1 = 0;
    TIM3_PSC = THERM_PSC;
    TIM3_ARR = THERM_PERIOD;
    TIM3_EGR = TIM_UG;
    TIM3_CR1 = TIM_CEN;
}

void set_therm(int x) {
    TIM3_CCR1 = x * THERM_PERIOD / 100;
}
```



Exercise

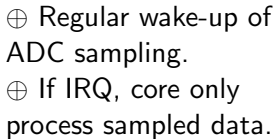
A servo-motor is a device with an arm which angle, relative to the body of the servo, can take an angle between 0 and 180. It is driven by PWM signal with a period of 20 *ms*. A position at 0 corresponds to a pulse width of 1 *ms* while a position of 180 corresponds to a pulse width of 2 *ms*.



It is not a good idea to have a pulse width at less than 1 *ms* because it will induces a time overhead on the next move. In the same way, having a pulse width ou of 2 *ms* may break the servo-motor.

- 1 We want to use TIM3 to drive the servo-motor on channel 1. Determine the est PSC and the period in pulses.
- 2 Write the initialization function `init_servo()`.
- 3 Write the function `set_servo(int n)` that set the position of the servo-motor at n° .
- 4 What is the precision of your setup in $^\circ$.





```

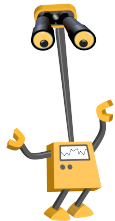
/* TIM3 */
TIM3_PSC = SAMPLE_PSC - 1;
TIM3_ARR = SAMPLE_DELAY;
TIM3_SR = 0;
TIM3_EGR = TIM_UG;
TIM3_CR1 = TIM_CEN;

```

```
/* IRQ */
void handle_sample() {
    int x = ADC1_DR;
    ...
    ADC1_SR = 0;
}
```

Outline

- 1 Introduction
- 2 Sensor / Actuators
- 3 Conclusion



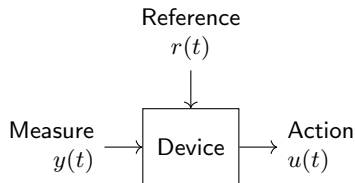
Linking Input and Output

Different physics units:

- ▶ input
- ▶ output

Example: line follower

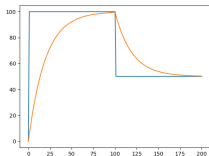
- ▶ IR: duration (PWM)
→ $[black, white]$
- ▶ speed in m/s → PWM



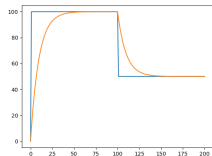
- ▶ regulation equations
- ▶ PID



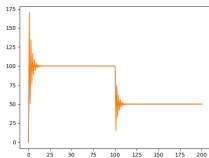
Proportionnnality



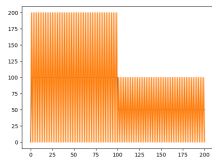
$$K_p = 0.05, K_i = 0, K_d = 0$$



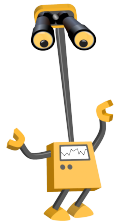
$$K_p = 0.1, K_i = 0, K_d = 0$$



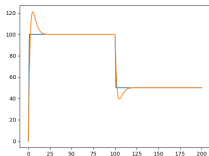
$$K_p = 1.7, K_i = 0, K_d = 0$$



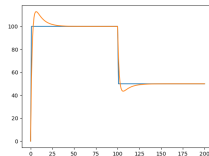
$$K_p = 2, K_i = 0, K_d = 0$$



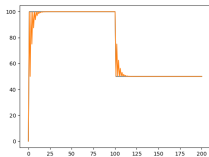
PI, PD,PID



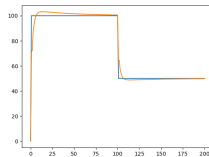
$$K_p = 0.5, K_i = 0.1, K_d = 0$$



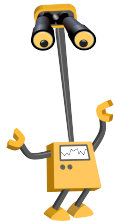
$$K_p = 0.5, K_i = 0.05, K_d = 0$$



$$K_p = 0.5, K_i = 0, K_d = 0.5$$



$$K_p = 0.5, K_i = 0.01, K_d = 0.2$$

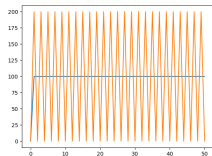


PID Tuning

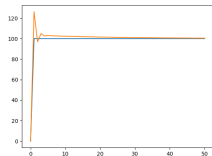
Lots of methods / driven device.

Ziegler-Nichols approach:

- ▶ set $K_i = 0$ and $K_d = 0$
- ▶ start with $K_p = 0$ and increase until oscillation
- ▶ $K_c = K_p$ and $P_c =$ period of oscillation
- ▶ $K_p = 0.6 \times K_c$
- ▶ $K_i = P_c/2$
- ▶ $K_d = P_c/8$



$$K_c = 2., P_c = .01 \text{ s}$$



$$K_p = 1.2, K_i = 0.05, K_d = 0.0125$$



Programming the PID

$dt = 1$ – same period

K_p, K_i, K_d time-dependent.

$$e(t) = r(t) - y(t)$$

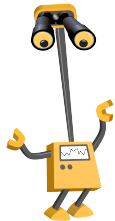
$$u(t) = K_p \times e(t) + K_i \times \int_0^t e(t) + K_p \times K_d \times (e(t) - e(t-1))$$

```

/* initialization */
#define Kp ...
#define Ki ...
#define Kd ...
int ep = 0;
int sum = 0;

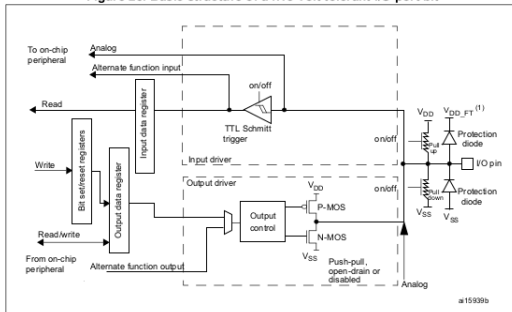
/* at each step */
int r = reference();
int y = sensor();
int e = r - y;
sum = sum + e;
u = Kp * e + Ki * sum + Kd * (e - ep);
ep = e;
set_actuator(u);

```



Multiplexing the pins

Figure 25. Basic structure of a five-volt tolerant I/O port bit



1 pin $\Rightarrow$ GPIO input/output, analog, alternate IO (+ MUX)

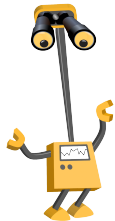
1 controller output/input $\Rightarrow n$ pins

Port	AF0	AF1	AF2	AF3	AF4	AF5	AF6	AF7	AF8	AF9	AF10	AF11	AF12	AF13	AF14	AF15
	SYS	TIM1/2	TIM3/4/5	TIM5/9/10/11	I2C1/2/3	SP11/9P12/I2S2/I2S5ext	SP13/I2Sext1/I2S3	USART1/2/3/I2S3ext	UART4/5/USART6	CAN1/2/TIM12/3/14	OTG_FS/OTG_HS	ETH	FSMC/SDIO/OTG_FS	DCMI		
PA0	-	TIM2_CH1_ETR	TIM5_CH1	TIM9_ETR	-	-	-	USART2_CTS	UART4_TX	-	-	ETH_MII_CRS	-	-	-	EVENTOUT
PA1	-	TIM2_CH2	TIM5_CH2	-	-	-	-	USART2_RTS	UART4_RX	-	-	ETH_MII_RX_CLK ETH_RMII_REF_CLK	-	-	-	EVENTOUT
PA2	-	TIM2_CH3	TIM5_CH3	TIM9_CH1	-	-	-	USART2_TX	-	-	-	ETH_MDIO	-	-	-	EVENTOUT
PA3	-	TIM2_CH4	TIM5_CH4	TIM9_CH2	-	-	-	USART2_RX	-	-	OTG_HS_ULPI_D0	ETH_MII_C0L	-	-	-	EVENTOUT
PA4	-	-	-	-	-	SP11_NSS	SP13_NSS I2S3_WS	USART2_CK	-	-	-	-	OTG_HS_SOF	DCMI_HSYNC	-	EVENTOUT
PA5	-	TIM2_CH1_ETR	-	TIM9_CHIN	-	SP11_SCK	-	-	-	-	OTG_HS_ULPI_CK	-	-	-	-	EVENTOUT
PA6	-	TIM1_BKIN	TIM3_CH1	TIM9_BKIN	-	SP11_MISO	-	-	-	TIM13_CH1	-	-	-	DCMI_PIXCLK	-	EVENTOUT

Complete System

Alternate Function Programming

- 1 set `GPIO_MODER_ALT` (alternate) in corresponding pin bits in `GPIOx_MODER`.
- 2 select the alternate function in
 - ▶ `GPIOx_AFR_L` (pins 0 to 7)
 - ▶ `GPIOx_AFR_H` (pins 8 to 15)(4 bit/pin)



Example: using TIM5_CH3 mapped to PA2 (alternate function 2)

```
GPIOA_MODER = REP_BITS(GPIOA_MODER, 2*2, 2, GPIO_MODER_ALT);
GPIOA_AFR_L = REP_BITS(GPIOA_AFR_L, 2*4, 4, 2);
```

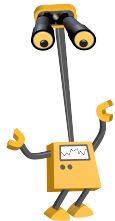

Connection map

Pin	Sensor/ Actuator	Controller	Alternate	Mode	Wire	Comments
PA2	Motor1	TIM5_CH3	2	PWM	blue	
		...				

Useful for:

- ▶ wire the device
- ▶ check if the wiring is consistent
- ▶ program the device
- ▶ check the wiring

Must be kept up to date!

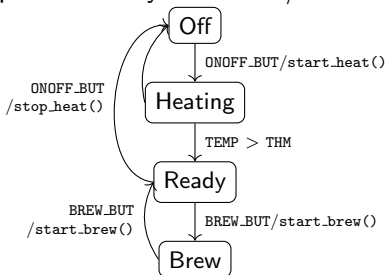


Modes

Definition (Mode)

A particular functioning arrangement or condition (Webster).

Implemented by automata/state diagram (UML/SysML):



State: variable (enumerated)

Events:

- ▶ bit change (SR)
- ▶ interrupt
- ▶ condition on input
- ▶ function call



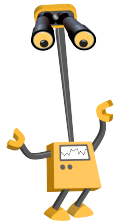
Several tasks

Time-triggered tasks:

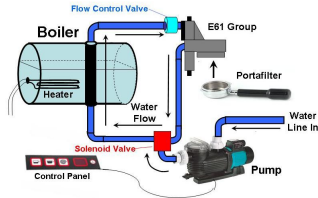
- ▶ frequency/period according to regulation rules
- ▶ state stored in global variables
- ▶ triggered by timers

Event-tasks:

- ▶ produced by sensors
- ▶ automaton (multiple, parallel, composed)
- ▶ triggered by status change/interrupt/other tasks



Exercise: driving the Espresso machine



Behaviour:

- 1 First, off (just wait).
- 2 When started (button On/Off), start heating.
- 3 When heat is enough, allows user to start brewing (button Brew). Heat is maintained.
- 4 When brewing is started, pressure in the hydraulic circuit is pushed to the right level (using pump motor and solenoid valve). Then the flow control valve is opened. Until brewing end, pressure and heat are maintained.
- 5 When button Brew is pressed once more, come back to ready state.



Questions (a)

Write the code to manage the user interface (buttons and LEDs) in the functions:

- 1 `init_ui()`
- 2 `set_onoff_led(int s)`
switch on/off the LED according to s (1/0).
- 3 `set_brew_led(int s)`
switch on/off the LED according to s (1/0).
- 4 `int test_onoff_button()`
return true if the button is pressed
- 5 `int test_brew_button()`
return true if the button is pressed



Questions (b)

Both pump motor and boiler heater use PWM signal produced by TIM3 with a period of 10ms.

Write the functions:

- 1 `init_TIM3()`
- 2 `start_heater()/stop_heater()`
- 3 `start_pump()/stop_pump()`
- 4 `set_heater(int n);/stop_heater(int n);` –
 $n \in [0, 100]$.

Then add the function for the valves (controlled by a MOSFET transistor):

- 1 `init_valves()`
- 2 `open_pump()/close_pump()`
- 3 `open_brew()/close_brew()`



Questions (c)

The ADC for boiler temperature and water pressure are started by TIM4 with a period of 100ms. They produce an interrupt that is handled to control the heater and the pump motor and pressure valve.

Both heater and pump are controlled by a PID with the constants: P_HEATER, I_HEATER, D_HEATER, P_PUMP, I_PUMP, D_PUMP. The reference temperature and pressure are defined by TH_HEATER and TH_PUMP.

- 1 Write `init_sensors()` that initializes TIM4 and ADC1/2.
- 2 Write `handle_adc` that handles ADC interrupts and call `heater` and `pump PID` with a new value:
 - ▶ `update_heater(int x)`
 - ▶ `update_pump(int x)`



Questions (d)

Write the main program.

- 1 Write the function that start and stop the PID use:
 - ▶ `start_heater()/stop_heater_pid()`
 - ▶ `start_pump()/stop_pump()`
- 2 Design an automaton to manage the Espresso application.
- 3 Write the `main()` program using the previously defined functions.

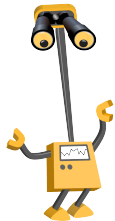
The following constants are available:

- ▶ BREW_TEMP, BREW_PRESS
- ▶ KP_HEATER, KI_HEATER, KD_HEATER
- ▶ KP_PUMP, KI_HEATER, KD_PUMP



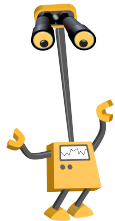
Outline

- 1 Introduction
- 2 Sensor / Actuators
- 3 Conclusion



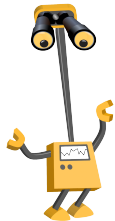
To sum up

- ▶ structure of a microcontroller
- ▶ programming inputs/outputs
 - ▶ infinite loop
 - ▶ polling approach
 - ▶ interrupt approach
- ▶ usual peripheral controllers
 - ▶ GPIO
 - ▶ DAC/ADC
 - ▶ timer
- ▶ how to put all together to make an application



To go forward

- ▶ communication
 - ▶ buses (USART, SPI, CAN, I^2C , ...)
 - ▶ wireless (Wifi, LoRa, SigFox, zigBee, RF, ...)
- ▶ energy management
 - ▶ clocking/not clocking the controllers
 - ▶ voltage/frequency scaling
 - ▶ sleep modes
 - ▶ wake-up on events
- ▶ DIY (Do It Yourself)
 - ▶ low prices of micro-controllers
 - ▶ (relatively) low prices of sensors/actuators
 - ▶ 3D printing, etc
 - ▶ FabLabs (CampusFab, F@bRiquet, Artilect)
 - ▶ lots of free available 3D models



Conclusion

